

AI TESTERS

Screenshot dla AI, decyzja dla człowieka

TESTY WIZUALNE W PLAYWRIGHCIE

`toHaveScreenshot()` działa świetnie, dopóki wzorce nie wylądują w repozytorium. Pokażę, jak przenieść je do Visual Regression Trackera i dołożyć warstwę, w której model ocenia **znaczenie** zmiany.

20 min

1 demo na żywo

0 plików PNG w repozytorium



Kto to mówi



Michał Ślęzak

Senior Test Architect w Sii · autor testingplus.me

- Buduje **AI Quality Gates**: obok testów funkcjonalnych stoją tam **VRT**, **ReportPortal**, Skills i agenci w pipeline.
- Prowadzi kurs **AI Testers: VRT z Playwrightem i TypeScriptem**, pięć sesji od pierwszego zrzutu do triage'u różnic przez model.
- Mówi na konferencjach testerskich, od **TestCon** w Stanach po **Nordic Testing Days**.

Playwright

Visual Regression Tracker

00 Agenda

Dwa akty, w każdym jedno pytanie.

I Co porównujemy?

6 min

01 Czym jest VRT

02 Poza testami funkcjonalnymi

03 Anatomia różnicy

04 toHaveScreenshot

II Gdzie jest wzorzec?

5 min

05 Visual Regression Tracker

 **Demo na żywo**

Obietnica: *wyjdiesz stąd, wiedząc, co zrobić z czterystu plikami PNG, których nikt nie ogląda.*

01 Czym jest VRT

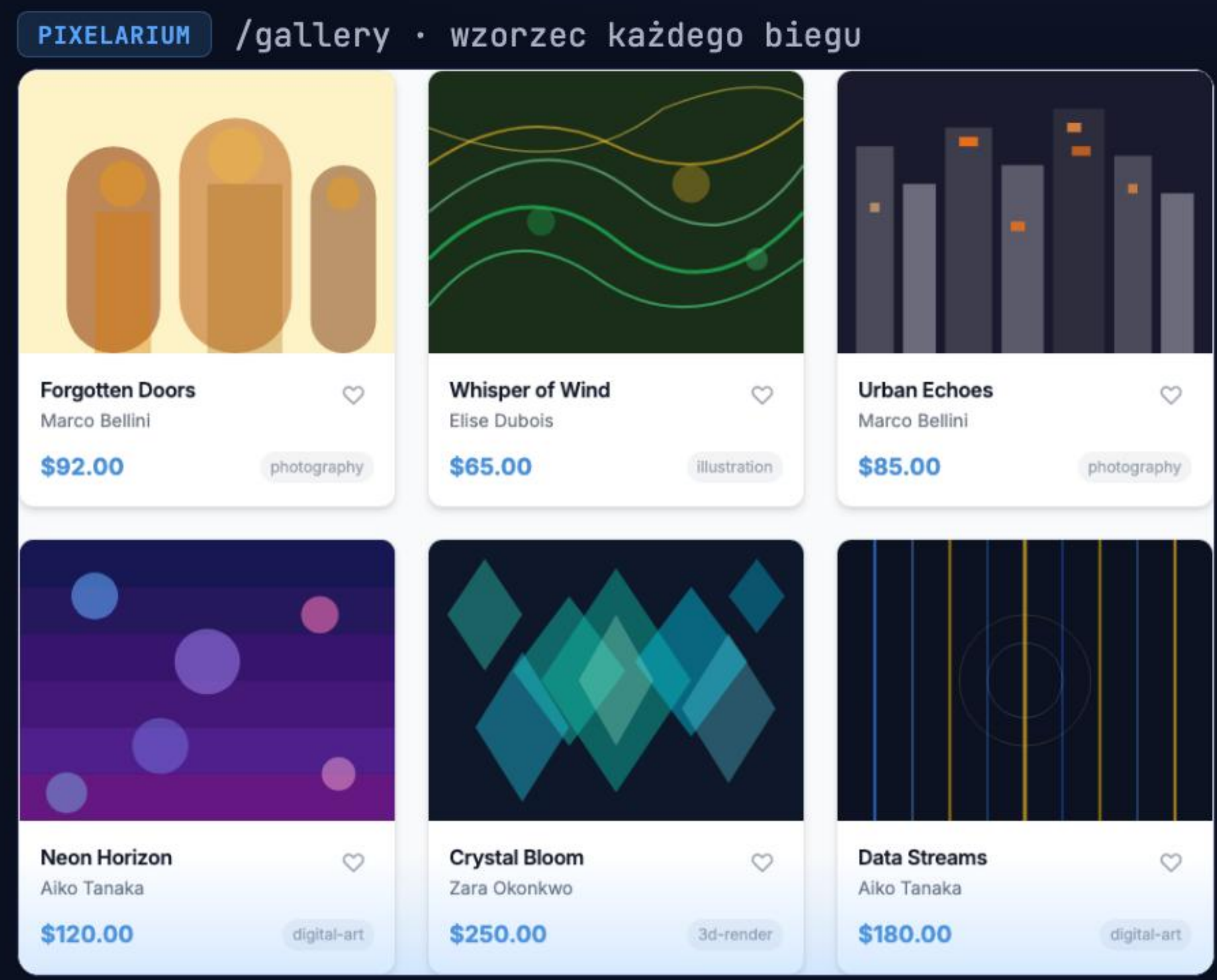
VISUAL REGRESSION TESTING

Definicja: VRT to testowanie regresji wizualnej. Test renderuje stronę, robi zrzut i porównuje go piksel po pikselu z zatwierdzonym wzorcem (baseline). Asercja dotyczy **wyglądu**, nie struktury DOM.

Test funkcjonalny potwierdzi, że przycisk istnieje i że da się go kliknąć. Nie zauważy, że przycisk zjechał pod stopkę, stracił kontrast albo że przykrył go baner cookie.

VRT nie zastępuje testów funkcjonalnych, tylko domyka lukę, której asercja na tekst nie wykryje.

924 × 699 px = **645 876** pikseli na każdy bieg



02 Poza testami funkcjonalnymi

Test funkcjonalny sprawdza, czy aplikacja **robi** to, co ma robić. Czy da się z niej korzystać, to inne pytanie.

FUNKCJONALNE Czy działa?

Asercja na stan, tekst, URL. Sygnał binarny.

Playwright Cypress

WIZUALNE · VRT Czy wygląda?

Diff pikseli jest **sygnałem**, nie diagnozą.

toHaveScreenshot Visual Regression Tracker

WYDAJNOŚĆ KLIENTA Czy jest szybko?

LCP, CLS, INP. Liczby są twarde, próg „za wolno” już nie.

 Lighthouse  Core Web Vitals

DOSTĘPNOŚĆ Czy da się obsłużyć?

Role ARIA, kontrast, kolejność fokusu. Skaner łapie regułę, sensu nie.

axe-core

Gdzie wizualne styka się z wydajnością: *CLS to regresja wizualna rozłożona w czasie. Zrzut jej nie złapie.*

GDZIE STOI PYTANIE

ASERCJA

OCENA

03 Anatomia różnicy

Playwright dokłada do raportu trzy obrazy. To one, nie komunikat błędu, mówią, co zaszło.

EXPECTED zatwierdzony wzorzec



ACTUAL bieżący bieg, po ?vrt=header-logo



DIFF mapa niezgodnych pikseli



CO Logo zniknęło z nagłówka. Reszta paska bez zmian.

GDZIE Lewy górny róg paska, ten, po którym poznaje się markę.

Sedno webinaru: diff mówi, że coś się zmieniło. Nie mówi, czy to źle. Ten sąd musi wydać ktoś inny.

04

Cztery decyzje w jednym teście. Każda z nich ratuje albo psuje powtarzalność biegów.

```
// 1. zegar zawsze PRZED goto, po nawigacji strona zdążyła odczytać Date.now()
await page.clock.setFixedTime(new Date('2026-01-15T12:00:00Z'));
await page.goto('/');

// 2. czekasz jawną asercją na element, nie na networkidle ani waitForTimeout
const hero = page.getByTestId('hero-section');
await expect(hero).toBeVisible();

await expect(hero).toHaveScreenshot('hero.png', {
  mask: [page.getByTestId('live-counter')], // 3. maska zachowuje układ
  maskColor: '#FF00FF',
  maxDiffPixels: 50, // 4. tolerancja liczbą, nie threshold
});
```

- 1 **Determinizm przed tolerancją.** Zamrożony zegar, wyłączone animacje, zamockowane API.
 - 2 **Czekaj asercją.** Nigdy `networkidle` ani `waitForTimeout` w teście, zawsze warunek na konkretny element.
 - 3 **Maskuj, nie ukrywaj.** Maska zachowuje układ, a `getByTestId` trzyma locator odporny na język.
 - 4 **Tolerancja liczbą.** `maxDiffPixels` mówi „ile”, `threshold` tylko rozmywa „co”.

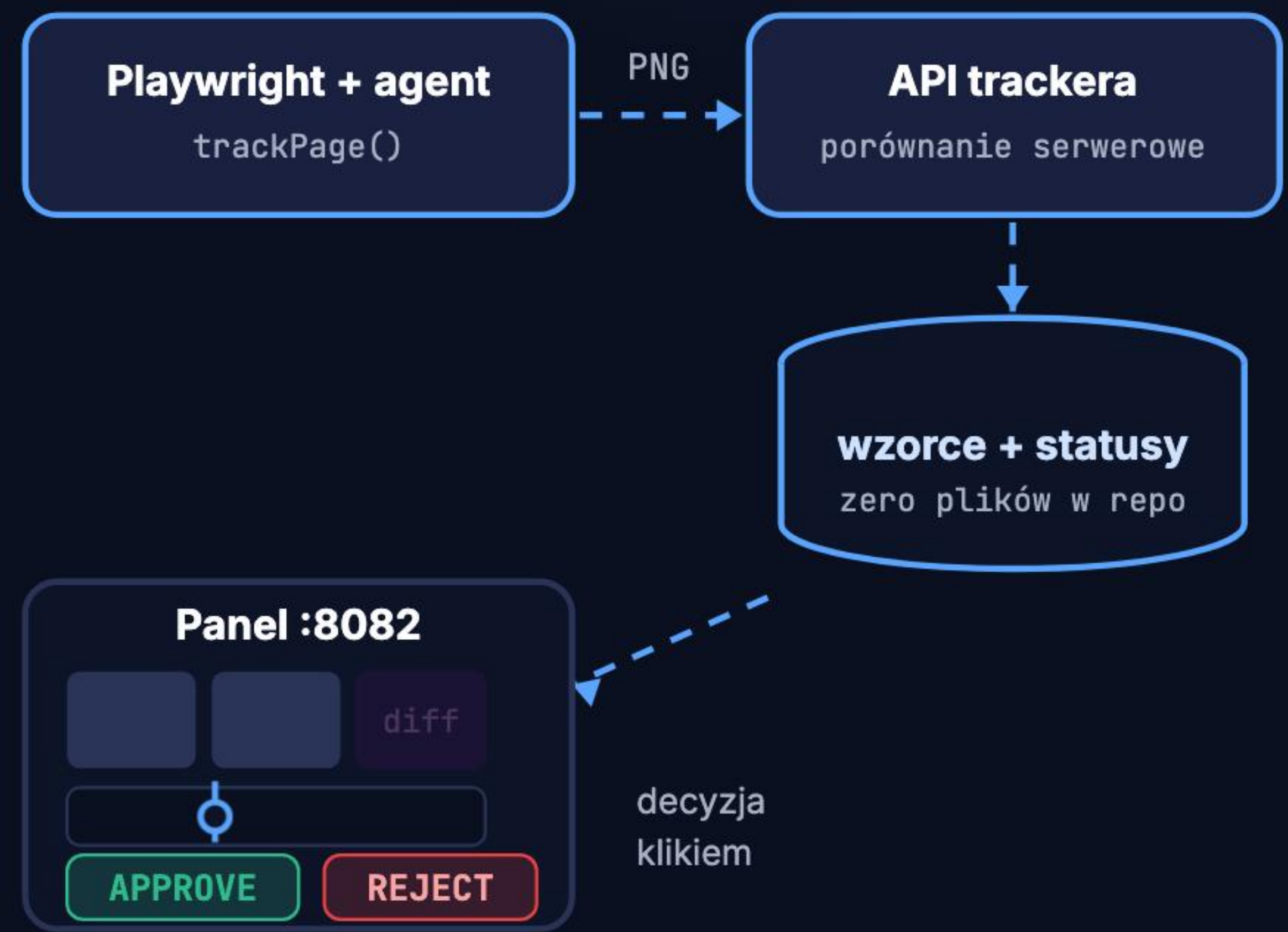
05 Visual Regression Tracker

Definicja: własny panel VRT w Dockerze. Wzorzec leży w bazie, porównanie robi serwer, status **zmienia człowiek klikiem**.

- W repozytorium testów **zero plików PNG**. Zatwierdzenie przestaje być commitem.
- Wariant to nazwa, przeglądarka, system, rozdzielczość i gałąź. Klucz ustala tracker, nie ścieżka pliku.
- Statusy i historię zatwierdzeń widzi cały zespół.

```
await vrt.trackPage(page, 'Homepage', {
  screenshotOptions: {
    mask: dynamicRegions,
    maskColor: '#FF00FF',
  },
  diffTollerancePercent: 1,
});
```

Wzorzec przestaje być plikiem w commicie. Staje się **stanem** z właścicielem i historią.



▶ Demo: VRT i triage promptem

● NA ŻYWO

Cztery kroki na żywym trackerze. Wzorzec omija repozytorium, ostatni krok to prompt.

1 Wzorzec łąduje w bazie

```
npm run demo:webinar
```

Konsola: `No baseline` i link do wpisu w panelu.
W repozytorium **zero plików PNG**.

2 Człowiek klika Approve

```
panel :8082 → Webinar Header → Approve
```

Klik w panelu zamiast commitu. Od tej chwili jest wzorzec,
a panel pamięta, kto go zatwierdził.

3 Wstrzykujemy regresję

```
npm run demo:webinar:bug
```

`?vrt=header-logo` gasi logo. Wpis dostaje status
`unresolved`, a diff widać w panelu.

4 Wklejamy prompt

```
PROMPT-VRT-TRIAGE.md → Claude Code
```

Claude sam pobiera wpisy `unresolved` z API, ogląda trójkę
expected, actual, diff i proponuje decyzję.